

SIH26002 — Backend PRD (v2)

AI-Based Smart Logistics and Accessibility Intelligence Platform for the North Eastern Region (NER)
Backend Product Requirements Document — Smart India Hackathon 2026 — reconciled with Frontend PRD (Saurabh) and AI/ML PRD (Omji)

1. Purpose & Scope

This document defines the backend requirements for SIH26002, updated after reconciling data fields with the Frontend PRD and integration points with the AI/ML PRD. It covers the data model, role-based access, API modules, and technical approach the backend team will build for the hackathon prototype.

2. Problem Summary

The North Eastern Region faces logistics and accessibility challenges from difficult terrain, extreme weather, limited transport connectivity, and frequent disruptions (landslides, floods, road damage). Essential goods — medicines, food, construction material, agricultural produce — are delayed to remote districts, and there is no integrated platform for real-time logistics visibility, accessibility status, predictive alerts, and route planning.

3. Backend Objectives

- Expose accessibility status of roads/bridges per district
- Ingest and serve vehicle location data for tracking essential-goods deliveries
- Generate candidate routes and expose them for AI-based ranking
- Generate automated alerts for blocked routes, high risk, and delayed deliveries
- Accept geo-tagged field reports, including from officers working offline
- Serve aggregated data to power dashboards (connectivity, bottlenecks, live status)
- Integrate with external weather APIs and the in-process AI/ML intelligence package

4. Tech Stack (Backend)

Component	Choice
Framework	Django + Django REST Framework
Database	PostgreSQL + PostGIS
Spatial ORM	GeoDjango
Async / scheduled tasks	Celery + Redis
AI/ML integration	In-process Python package (scikit-learn) called from Django services — no separate microservice
Location tracking	REST ping from the driver's mobile app (no dedicated GPS hardware)
Routing engine	Backend graph logic (candidate routes) + AI ranking layer
Weather data	Periodic pull from a public weather API via Celery

Media storage	Local storage for the MVP demo
i18n	Django's built-in translation framework for alert/notification templates

5. Backend Modules (Django Apps)

Kept as a single modular Django project — not split into microservices — to stay within hackathon scope.

App	Focus	Notes
accounts	Auth & roles	DRF token/JWT auth; Admin, Field Officer, Normal User roles.
routes	GIS core	Infrastructure & District models (GeoDjango); accessibility status per district/road; candidate-route generation (graph logic).
intelligence	AI integration	Python package called directly from Django views/services (no separate microservice) — wraps Omji's risk classifier, ETA regressor, and route ranker.
vehicles	GPS tracking	Vehicle & LocationPing models; ingest periodic pings from the driver's app; latest-location API.
alerts	Notifications	Celery + Redis periodic task checks conditions and creates Alert rows; combines AI outputs with rule triggers.
reports	Field reporting	IncidentReport model with photo + geo-tag; local media storage.
sync	Offline sync	Single batch endpoint accepting cached records with client timestamps; insert-if-not-exists, ordered by client_timestamp.
dashboard	Aggregation APIs	Read-only viewsets/serializers over existing models — no new heavy models.

6. Roles & Permissions

Three roles, stored on Profile.role: Admin, Field Officer, Normal User (covers both the individual/public persona and the transporter persona).

Model	Admin	Field Officer	Normal User
Profile	CRUD (all)	R/U (own only)	R/U (own only)
District	CRUD	R only	R only
Infrastructure	CRUD (status/condition/risk)	R only	R only
Vehicle	CRUD (all, fleet view)	No access	CRUD (own vehicle only)
LocationPing	R only (monitoring)	No access	C + R (own vehicle only)
Trip	CRUD (all, can override)	No access	CRUD (own trips only)
Alert	CRUD (create/resolve/publish)	R only (own area)	R only
IncidentReport	CRUD (review/resolve)	C + R (own reports)	No access (MVP scope)
WeatherSnapshot	R only (system-written)	R only	R only

RouteCandidate (computed)	R (analytics)	Not applicable	R (primary consumer)
---------------------------	---------------	----------------	----------------------

Field Officers cannot edit Infrastructure directly — they submit IncidentReports, and Infrastructure status is updated through that validated path (by the system or Admin) to protect data integrity. IncidentReport is out of scope for Normal User at MVP — the problem statement mandates field officials and authorities, not open public reporting.

7. Database Schema

Nine models cover the full requirement set (accessibility monitoring, tracking, alerts, field reports, offline sync) plus the fields needed for AI integration and frontend data-contract alignment.

Profile

Extends Django's built-in User to capture role and contact info.

Field	Type	Notes
role	CharField (choices)	admin / field_officer / normal_user
phone	CharField	Contact number

District

Administrative district boundary, connectivity status, and accessibility breakdown for the accessibility map/heatmap.

Field	Type	Notes
name	CharField	District name
geom	PolygonField (GeoDjango)	District boundary
connectivity_status	CharField (choices)	good / moderate / poor
accessibility_score	FloatField (0–10)	Overall accessibility score
road_condition_score	FloatField	Breakdown component
healthcare_access_score	FloatField	Breakdown component
market_access_score	FloatField	Breakdown component
wheelchair_access	BooleanField (nullable)	Breakdown component, optional

Infrastructure

Roads and bridges — a single model with a type flag, not two separate tables. Carries AI-facing risk fields.

Field	Type	Notes
name	CharField	Road / bridge name
type	CharField (choices)	road / bridge
geom	LineStringField (GeoDjango)	Path geometry
district	ForeignKey → District	Owning district

status	CharField (choices)	accessible / blocked / risky
condition	CharField (choices)	good / moderate / poor / damaged — AI risk-model feature
risk_score	FloatField	Operational risk score
disruption_probability	FloatField (0–1)	AI-01 model output
risk_level	CharField (choices)	low / moderate / high / critical
top_factors	TextField	AI explainability (e.g. “heavy rainfall; poor road condition”)
last_updated	DateTimeField (auto_now)	Last status change

Vehicle

A vehicle carrying essential goods, tracked via the driver's phone app.

Field	Type	Notes
registration_no	CharField (unique)	Vehicle registration number — exposed to frontend as “id”
cargo_type	CharField (choices)	medicine / food / construction / agri_produce
driver_name	CharField	Driver's name
driver_phone	CharField	Driver's contact number
status	CharField (choices)	active / idle — vehicle availability, not trip progress

LocationPing

GPS location history, posted periodically by the driver's mobile app — no separate hardware GPS module.

Field	Type	Notes
vehicle	ForeignKey → Vehicle	Reporting vehicle
lat	FloatField	Latitude
lng	FloatField	Longitude
location_name	CharField (nullable)	Reverse-geocoded place name, optional
speed	FloatField (nullable)	AI-02 ETA model feature
timestamp	DateTimeField	Time of the ping

Trip

A single delivery run for a vehicle. Carries both the planned and AI-predicted ETA separately.

Field	Type	Notes
vehicle	ForeignKey → Vehicle	Assigned vehicle
origin	CharField / PointField	Start point

destination	CharField / PointField	End point
start_time	DateTimeField	Trip start
end_time	DateTimeField (nullable)	Trip end
status	CharField (choices)	on_route / delayed / at_risk / delivered
base_eta	DateTimeField	Planned/base ETA
predicted_eta	DateTimeField (nullable)	AI-02 model output
expected_delay	DurationField (nullable)	AI-02 model output

Alert

Automated alert raised for blocked roads, disruptions, or delayed deliveries.

Field	Type	Notes
type	CharField (choices)	blocked_road / flood / landslide / delay / traffic / weather / safety
infrastructure	ForeignKey → Infrastructure (nullable)	Related road/bridge, if any
trip	ForeignKey → Trip (nullable)	Related trip, if any
severity	CharField (choices)	low / moderate / high / critical
title	CharField	Short headline for list/notification views
message	TextField	Full description
recommended_action	CharField	e.g. “Use alternate route via ...”
created_at	DateTimeField (auto_now_add)	Raised at
resolved	BooleanField	Whether the alert is closed

IncidentReport

Geo-tagged field officer report, including photos. Primary model exercised by offline sync and by AI-08 (field intelligence input).

Field	Type	Notes
officer	ForeignKey → User	Reporting field officer
geom	PointField (GeoDjango)	Report location
photo	ImageField	Uploaded photo
description	TextField	Officer's free-text notes
incident_type	CharField (choices)	Structured type — AI feature input
severity	CharField (choices)	low / moderate / high / critical — AI feature input

client_timestamp	DateTimeField	Captured on-device (may be offline)
synced_at	DateTimeField	Received by server — kept separate for sync ordering

WeatherSnapshot

Periodic weather-API pulls, feeding both the risk-score calculation and the AI-01 risk model.

Field	Type	Notes
district	ForeignKey → District	District the reading applies to
rainfall_mm	FloatField	Rainfall reading
condition	CharField (choices)	clear / moderate / heavy / extreme
recorded_at	DateTimeField	Reading timestamp

RouteCandidate (computed response — not a stored model)

Backend graph logic generates 2–3 candidate routes from Infrastructure data; the AI intelligence package scores and ranks them. This shape is returned by the route API, never persisted as-is.

Field	Type	Notes
route_id	string	Generated per request, not stored
distance_km	float	From backend routing/graph logic
base_eta	int (minutes)	From backend routing/graph logic
traffic_level	low / moderate / high	Current condition
risk_level	low / moderate / high / critical	From AI-01
accessibility_score	float (0–10)	From AI-04 / District data
recommended	boolean	Set by AI-03 ranking
polyline	string (encoded path)	For map rendering
legs[]	array (optional)	Multimodal support — empty/1-item at MVP (road-only)
top_factors / explanation	string	AI explainability for the ranking

8. Location Tracking Approach

Vehicles are tracked through the driver's own mobile app rather than a dedicated hardware GPS module, since an app is already being built for this project.

- The app reads device location periodically and POSTs it with vehicle_id, lat, lng, speed, and timestamp
- The backend stores each ping in LocationPing and exposes a “latest location” endpoint per vehicle
- No hardware integration or IoT protocol (e.g. MQTT) is required for this flow

9. Offline Sync

Field officers and drivers in low- or no-network areas need to keep working without connectivity.

- The mobile app caches records locally (location pings, incident reports) while offline
- Once connectivity returns, the app sends all cached records to a single batch-sync endpoint
- The backend inserts records if they don't already exist and orders them by their original `client_timestamp`
- Conflict handling is kept simple: last-write-wins — no custom merge logic

10. AI/ML Integration Notes

Per the AI & Intelligence Engine PRD, the intelligence layer runs in-process as a Python package, not a separate service.

- Risk prediction (AI-01) and ETA/delay prediction (AI-02) consume Infrastructure, WeatherSnapshot, and IncidentReport fields listed above
- Route ranking (AI-03) consumes backend-generated RouteCandidate objects and returns them re-ordered with a recommended flag and explanation
- Accessibility intelligence (AI-04) reads/writes the District accessibility fields
- Alert intelligence (AI-06) combines these model outputs with the rule-based Celery triggers already defined in the alerts app

11. Out of Scope (Deliberately Excluded)

To avoid over-engineering the hackathon prototype, the following are explicitly excluded from the backend build:

- Custom deep-learning model training, LSTM/complex time-series models — Random Forest classifier/regressor only (per AI PRD)
- Agentic AI / LLM chatbot functionality — explicitly excluded from this MVP
- Django Channels / WebSockets for real-time push — REST polling (10–15s) is enough for the MVP
- Microservices or Kubernetes, including a separate AI microservice — single Django monolith, AI runs as an in-process Python package
- Custom conflict-resolution logic for offline sync — last-write-wins by `client_timestamp`
- Dedicated hardware GPS modules — location comes from the driver's mobile app
- Cloud object storage (S3) and multi-cloud setup — local media storage is sufficient for the demo
- Mandatory live satellite-data ingestion pipeline — GeoAI flood layer is optional/bonus with a static sample-image fallback

12. Team & Ownership

Member(s)	Responsibility
Omji	AI / ML
Arpit	Backend — API management, database management
Ayush, Rahul	Web frontend
Radhika, Saurabh	App frontend

13. Open Items

- Confirm which public weather API/data source will be used for rainfall thresholds
- Confirm routing engine (self-hosted OSRM vs. third-party Directions API) for candidate-route generation
- Finalize alert severity thresholds with the AI/ML side (Omji) once the risk-score formula is set
- Confirm whether Field Officer app screens are in scope with Saurabh (frontend currently has no dedicated screen for this role)